Lab 1 - Secure Software Development with Rust

Cybersecurity & Infrastructure | Student Lab Guide | Security by Design

What this lab is about

In this lab you will be asked to think carefully about simple software designs, you will not be asked to write complicated software. A program is only as reliable as the assumptions made while designing it. Your goal is to identify those assumptions, test them, and strengthen the program so that unexpected or malicious input does not cause unsafe or unpredictable behavior.

The principle we will use throughout these labs is security by design. Instead of waiting for a weakness to be exploited and reacting afterwards, we make intelligent design choices that remove/reduce weaknesses before the software is deployed.

Learning outcomes

By the end of this session, you should be able to create and run a simple Rust project, recognize unsafe assumptions in user input, test a program with unexpected values, handle invalid input without crashing, and explain why your changes make the program more robust.

1. Prerequisites and installation

You will use **Visual Studio Code** as the editor and the standard Rust toolchain for compiling and running code.

Step 1 - Install Visual Studio Code

Download and install Visual Studio Code for your operating system. Open VS Code once installation is complete.

Step 2 - Install Rust

Install Rust using the official **rustup** installer for your operating system. The installation should include **rustc** (the Rust compiler), **Cargo** (Rust's project/build tool), and rustup (toolchain management). Use the default stable toolchain unless instructed otherwise.

Step 3 - Install rust-analyzer

Open VS Code, open the Extensions panel, search for **rust-analyzer**, and install the extension. This provides Rust-aware code completion, diagnostics, type information, and useful compiler feedback inside VS Code.

Step 4 - Verify your installation

Open a terminal and run the following commands. Each command should return version information:

```
rustc --version
cargo --version
```

If either command is not recognized, do not continue by running code. Fix the installation first or ask for assistance.

Step 5 - Create your first project

create a new folder and open that folder in VS code.

In the terminal, create and run a new Rust project:

```
cargo new secure_lab_1
cd secure_lab_1

cargo run
```

Open the **secure_lab_1** folder in VS Code. Your main program will be in **src/main.rs**.

2. Your Session 1 Rust toolbox

For this exercise you are expected to work using the concepts below. Do not replace the exercise with advanced libraries or techniques that you cannot explain.

Tool	What you may use it for
`let / mut`	Store values and make a value mutable only when necessary.
Basic types	Represent data using an appropriate and predictable type.
`if / else`	Enforce conditions, ranges, and simple rules.
Functions	Separate a clear task such as validation from the rest of the program.
`trim()`	Remove surrounding whitespace from text input.
`parse()`	Attempt to convert text into another type.
`Result` and `match`	Handle success and failure explicitly instead of assuming success.

3. Exercise - Break it, then strengthen it

Scenario: The program below asks the user for their age and grants access when the entered age is 18 or above. The programmer has assumed that the user will always enter a valid number.

```
use std::io;

fn main() {
    println!("Enter your age:");
    let mut input = String::new();
    io::stdin().read_line(&mut input).unwrap();

    let age: i32 = input.trim().parse().unwrap();

    if age >= 18 {
        println!("Access granted");
    } else {
        println!("Access denied");
    }
}
```

Part A - Understand the normal behavior

Run the program with normal inputs such as 20 and 15. Before changing anything, make sure you understand what every important line is doing.

Part B - Stop behaving like the expected user

Try to make the program fail or behave incorrectly. Test at least four inputs that challenge the programmer's assumptions. Examples may include text instead of a number, an empty input, a negative value, or an unrealistic value. Record what you tested and what happened.

Part C - Identify the weakness

Answer these questions before changing the code: **What assumption did the programmer make? Which line makes that assumption dangerous? What should the program do instead when the input is invalid?**

Part D - Strengthen the program

Modify the program so invalid input is handled safely rather than causing the program to terminate unexpectedly. Use parse() together with Result/match. Then add a sensible validity rule for age. Your program should clearly tell the user when the supplied value is not accepted.

Part E - Attack your strengthened version

Repeat your earlier tests from Part B against the new version. your goal is to show that the program now behaves predictably when the user does something unexpected.

4. What you must be able to explain

During the lab, you may be asked questions about your own implementation. You should be able to explain what you changed and why.

- What input caused the original program to fail?
- Why was unwrap() a problem in this situation?
- What does parse() return when conversion succeeds or fails?
- Why did you use match here?
- What happens if your range check is removed?
- Give me an input your program rejects and explain why.
- What assumption does your strengthened program still make?

5. Use of AI

AI is permitted and may be used as a learning tool. You are not being assessed on whether an AI system can produce working Rust code. You are being assessed on whether **you** understand and can defend the work you submit.

When instructed, begin your AI conversation using the standard lab starting prompt. Continue the conversation as needed while working. At the end, use the provided export procedure and submit the required conversation record with your work. Do not rely on AI-generated techniques outside the Session 1 toolbox unless you can explain them and have received permission to use them.

If AI writes a line of code that you submit, that line becomes your responsibility. You may be asked what it does, why it is there, what would happen if it were removed, or how you would modify it.

Standard AI starting prompt

I am completing Lab 1: Secure Software Development with Rust. Act as a learning assistant, not as a replacement for my work. For this lab, keep all guidance within this toolbox unless I explicitly ask for an explanation of something else: let/mut, basic types, if/else, functions, trim(), parse(), Result, and match. Do not introduce advanced libraries, crates, or techniques as solutions. Help me understand errors, test assumptions, and reason about how to strengthen the program. If I ask for code, explain the purpose of the important lines so that I can understand and defend them.

Standard AI export prompt

Create an end-of-lab record of this conversation. Summarize: (1) the problems or weaknesses I asked about, (2) the test cases I discussed, (3) the changes or solutions you suggested, and (4) any Rust concepts you explained. Do not add new solutions or improve my work in this response. Clearly separate what I asked or attempted from what you suggested. After the summary, provide the complete conversation in an exportable format if the platform supports it; otherwise instruct me to use the platform's available share/export feature and submit the conversation as instructed by my lab teacher.

6. Submission checklist

Before submitting, make sure you have: a working Rust project; your strengthened **src/main.rs**; evidence/notes for at least four original test inputs plus one additional test after hardening; short answers identifying the original assumption and the reason your new design is safer; and the required AI conversation record if AI was used or if the instructor requires it.

7. The idea to take away

Security is not only about detecting attacks after they happen. Secure software development asks what can be prevented during design. In this lab, a single assumption - that the user will enter the expected input - was enough to make the program fragile. Your job as a developer is to identify assumptions like this and decide which ones can be removed.